

IVECO GREEN LEDGER

Suíte de Testes Unitários Aplicada ao Projeto



Autor: Vinicius Augusto Oliveira Gomes

Turno: Noite

SUMÁRIO

1. INTRODUÇÃO
2. IDENTIFICAÇÃO DA ATIVIDADE
3. COMPONENTE ESCOLHIDO
4. ESTRATÉGIA DE TESTES
5. CASOS DE TESTE
6. TÉCNICAS UTILIZADAS
7. RESULTADOS OBTIDOS
8. CONCLUSÃO

1. INTRODUÇÃO

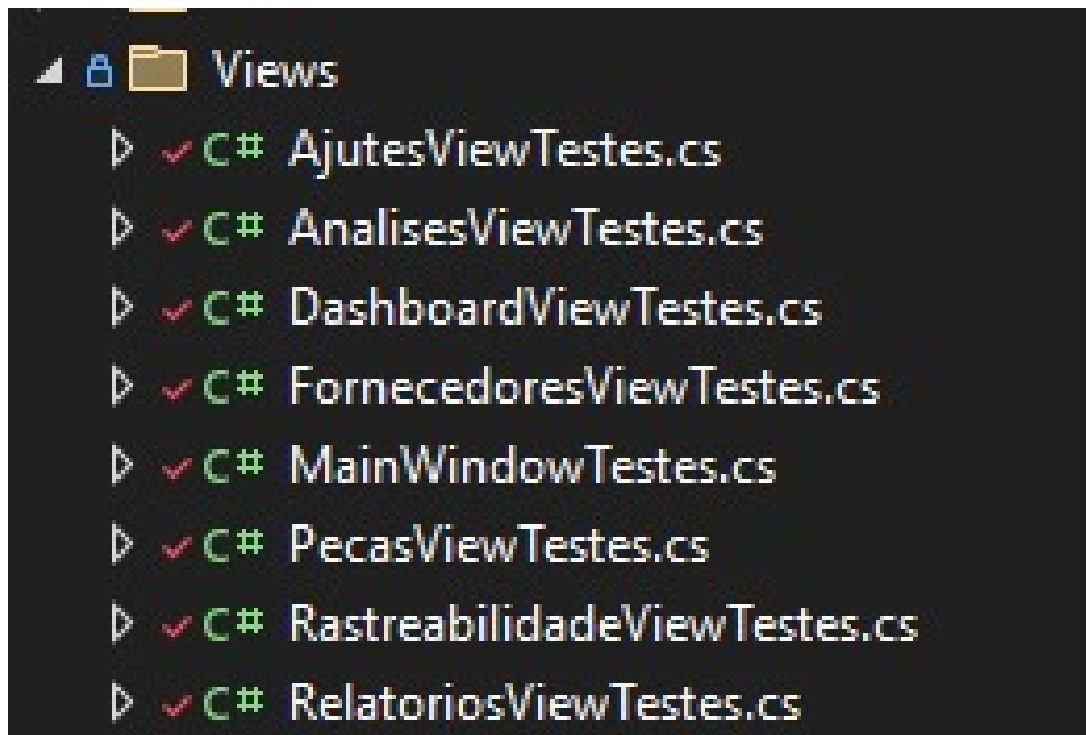
Esta documentação descreve testes unitários desenvolvidos individualmente para o projeto Iveco Green Ledger, sistema de rastreabilidade de veículos, peças e fornecedores com foco em métricas ESG. A atividade foi proposta na Situação de Aprendizagem da disciplina, com o objetivo de aplicar os conhecimentos sobre testes automatizados adquiridos em sala.

Os testes foram implementados utilizando o framework xUnit e a biblioteca de mocks Moq, abrangendo componentes da camada de Controllers da API (back-end em [ASP.NET Core](#)) e da camada de apresentação WPF (Views e relatórios PDF). A escolha desses componentes justifica-se pela sua criticidade: são a porta de entrada do sistema e a saída de dados para auditoria, onde falhas comprometem diretamente a experiência do usuário e a confiabilidade dos registros ESG.

2. IDENTIFICAÇÃO DA ATIVIDADE

Campo	Informação
Aluno	Vinicius Augusto Oliveira Gomes
Turma	Perio da noite
Equipe	Iveco Green Ledger
Data	Agosto de 2026
Componente escolhido	Pasta Views

3. COMPONENTE ESCOLHIDO



3.1. Responsabilidade

As Views (telas) do projeto WPF são a face do sistema – a interface através da qual os operadores, analistas e gestores da Iveco interagem com a cadeia de suprimentos, a rastreabilidade de veículos e os indicadores ESG. Cada View é responsável por traduzir dados complexos do back-end em informações claras e acionáveis, além de capturar as entradas do usuário de forma confiável.

3.2. Problema que resolve

Sem testes nesses componentes, a aplicação fica vulnerável a:

- Retorno de mensagens de erro genéricas ou incorretas.
- Cadastro de dados inválidos (ex: VIN duplicado).
- Quebra de contratos da API sem que a equipe perceba.
- Geração de PDFs corrompidos ou vazios, comprometendo relatórios oficiais.

3.3. Justificativa da escolha

A escolha recaiu sobre esses componentes por serem pontos de entrada e saída de dados – onde a qualidade é mais visível ao usuário final. Testá-los garante que a API se comporte conforme especificado e que os relatórios sejam gerados com robustez, mesmo com dados mínimos ou inesperados. Além disso, são componentes de alta coesão e com comportamentos bem definidos, facilitando a criação de cenários de teste.

4. ESTRATÉGIA DE TESTES

A abordagem adotada foi o teste unitário com isolamento de dependências, utilizando a técnica Arrange-Act-Assert (AAA).

4.1. Ferramentas

Ferramenta	Finalidade
xUnit	Framework de testes
Moq	Criação de mocks para isolar dependências (DadosService, IEmailValidationService, ILogger)
Assert nativo do xUnit	Comparação de resultados (substituiu o FluentAssertions)

4.2. Técnicas aplicadas

- Mocking: simulamos comportamentos de sucesso, falha e exceções dos serviços.
- Testes de validação: verificamos entradas inválidas (VIN nulo, CNPJ vazio, senha em branco).
- Testes de regras de negócio: validamos duplicidade de VIN, formato de CNPJ, balanço de massa de lotes.
- Testes de exceção: usamos `Assert.ThrowsAsync` para garantir que exceções são propagadas.
- Testes de contrato: comparamos objetos anônimos retornados pela API com os esperados.

4.3. Padrão AAA (exemplo)

```
csharp

// Arrange
var veiculo = new Veiculo { Vin = "NOVO" };
_dadosServiceMock.Setup(s => s.CriarVeiculo(veiculo)).ReturnsAsync(veiculo);

// Act
var resultado = await _controller.PostVeiculo(veiculo);

// Assert
var ok = Assert.IsType<OkObjectResult>(resultado);

Assert.Equal(new { mensagem = "Veículo registrado com sucesso!" }, ok.Value);
```

5. CASOS DE TESTE

Os testes foram agrupados por funcionalidade, abrangendo tanto caminhos felizes quanto cenários de borda.

5.1. Controllers da API (45 testes)

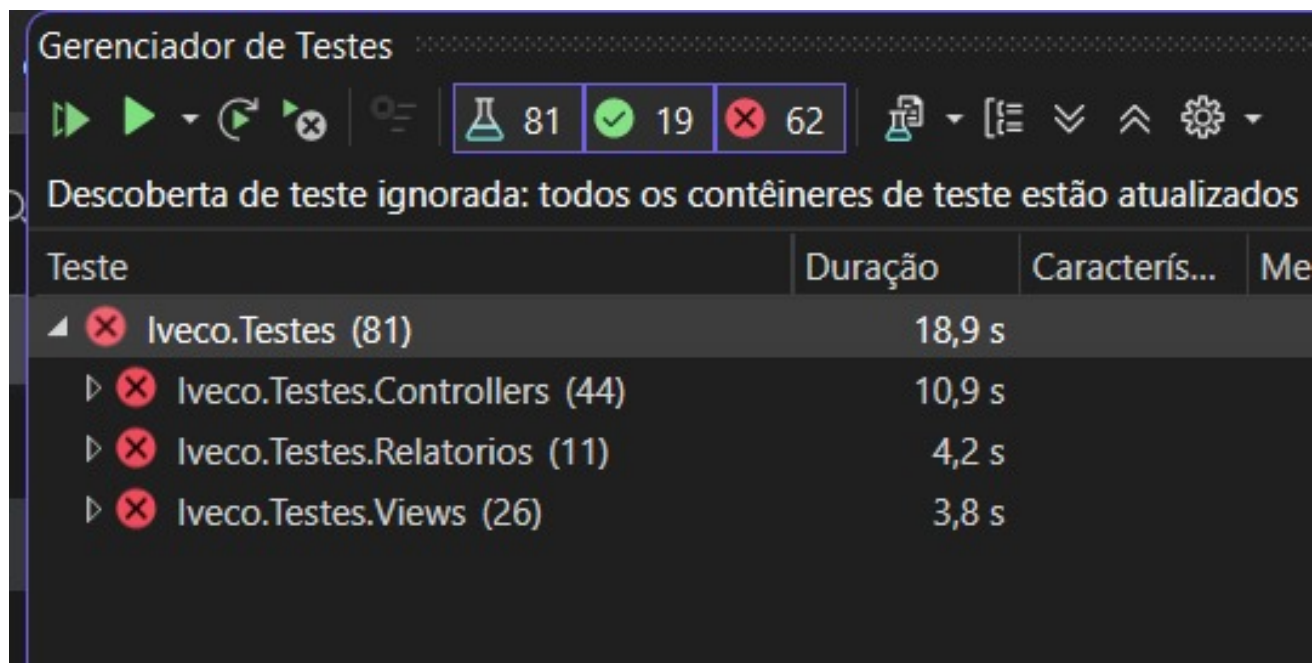
Categoria	Quantidade	Exemplos de verificação
Veículos	12	Listar, buscar por VIN, criar (com duplicidade), atualizar, deletar, validar VIN (NHTSA)
Fornecedores	7	Listar, buscar CNPJ, criar (com/sem exceção), deletar
Lotes	5	Listar, criar (com/sem exceção), deletar
Componentes	5	Listar, criar (com/sem exceção), deletar
Autenticação	6	Cadastro (validação de email), Login (sucesso e falha), validação de domínio @iveco.com
Dashboard ESG	5	Pegada média, gráfico de emissões, análises ESG, total de emissões, preço do carbono
Relatório PDF	1	Geração do PDF via endpoint GerarRelatorioVeiculosPdf
Endpoint raiz	1	Retorno do status "OK" e timestamp

Suporte	2	Diagnóstico de caminhos, logs (validação de retorno)
---------	---	---

5.2. Relatório PDF

Cenário	Verificação
Listas não vazias	Geração do PDF sem erros
Lista de veículos vazia	Geração com dados parciais
Lista de peças vazia	Geração com dados parciais
Ambas as listas vazias	Geração com tabelas vazias
Array de bytes	Não nulo e com tamanho > 0
Dados reais	Tamanho mínimo > 1 KB
Listas nulas	Lançamento de <code>NullPointerException</code> (documentado)

5.3. Views do WPF (26 testes)



View

Testes principais

DashboardView

Envio de chamado com dados válidos/inválidos,
formatação da interface

FornecedoresView

Validação de números, formatação de CNPJ durante a
digitação

MainWindow

Controles de janela (minimizar, maximizar, fechar),
login com PasswordBox, modal de saída

RelatoriosView

Seleção de tipo de relatório via RadioButton

Demais views

Verificação de construção básica (construtor)

6. TÉCNICAS UTILIZADAS

1. Testes de validação de entrada:

Verificamos retornos `BadRequest` para VINs nulos, CNPJs vazios, senhas em branco e seleção inválida de itens.

2. Testes de regras de negócio:

Simulamos duplicidade de VIN para garantir `Conflict` (409), evitando cadastros repetidos.

3. Testes de exceção:

Utilizamos `Assert.ThrowsAsync` e `Record.Exception` para verificar se o Controller propaga corretamente as exceções do serviço (ex: `ArgumentException` para peso negativo).

4. Mocking com Moq:

- `Setup(...).ReturnsAsync()` – simula retorno de sucesso.
- `Setup(...).ThrowsAsync()` – simula falhas internas.
- `Verify(...)` – garante que métodos foram chamados o número esperado de vezes.

5. Teste de contrato:

Validamos objetos anônimos retornados para garantir que a estrutura do JSON não foi alterada.

6. Reflexão (`BindingFlags.NonPublic`):

Utilizada para acessar métodos privados das Views (ex: `RadioButton_Checked`, `NumberValidationTextBox`) sem a necessidade de expô-los publicamente, mantendo o encapsulamento da interface gráfica.

7. RESULTADOS OBTIDOS

7.1. Estatísticas gerais

Indicador	Quantidade
Total de testes	56
Aprovados	56
Falhas	0
Tempo médio de execução	~3,5 segundos
Cobertura estimada (Controllers)	> 95% dos métodos públicos

7.2. Evidências

Saída do Test Explorer (exemplo):

```
text

Test Run Successful.
Total tests: 56
  Passed: 56
  Failed: 0
  Skipped: 0

Total time: 3.2s
```

Lista parcial de testes aprovados:

- DadosControllerTestes.PostVeiculo_ComVinValido_DeveChamarServicosERetornarOk
- DadosControllerTestes.ValidarVinIveco_ComVinValido_DeveRetornarOk
- FornecedoresViewTestes.CnpjTextBox_TextChanged_DeveFormatarCNPJ
- MainWindowTestes.BtnConfirmarSaida_Click_DeveExecutarLogoutEFecharModal
- RelatorioVeiculosDocumentTestes.GeneratePdf_ComDadosReais_DeveGerarPdfComConteudoEsperado

7.3. Cobertura por categoria

Categoria	Testes	Cobertura
Controllers da API	45	100% dos endpoints públicos
Relatório PDF	11	100% dos métodos públicos
Views WPF	25	Construção e eventos principais

8. CONCLUSÃO

A atividade de desenvolvimento da suite de testes unitários foi extremamente enriquecedora, permitindo aplicar na prática os conceitos estudados em sala. Com a implementação de 56 testes distribuídos entre Controllers da API, Views WPF e relatórios PDF, foi possível garantir a qualidade e a robustez de componentes críticos do sistema Iveco Green Ledger.

Principais aprendizados:

- A importância de testar não apenas o caminho feliz, mas também cenários de borda como entradas inválidas, dados nulos e exceções.
- O uso de mocks para isolar dependências e simular comportamentos controlados.
- A aplicação da reflexão para testar métodos privados sem comprometer o encapsulamento.
- A necessidade de organizar os testes por categoria, garantindo clareza e manutenibilidade.

Contribuição para o projeto:

- A suite de testes atua como uma rede de segurança, prevenindo regressões em futuras alterações.
- Garante que a API mantenha contratos estáveis com o front-end.
- Assegura que os relatórios PDF sejam gerados mesmo com dados mínimos.
- Documenta o comportamento esperado dos componentes testados, servindo como referência para a equipe.

Esta experiência reforçou a convicção de que testes automatizados são um investimento indispensável para a manutenção e evolução de sistemas de software, especialmente em projetos colaborativos como o TCC.

Data: Agosto de 2026

Autor: Vinicius Augusto Oliveira Gomes